

Contextual Auth בתוך Audit – Email Auth

085aa23 · HEAD · 26.09.2026 · Teamder · עץ נקי

נקרא ישירות מהקוד. לא שונה קוד, לא בוצע deploy, לא נגעתי ב-Firebase rules או ב-enableImprovedEmailPrivacy. עצירה לפני מימוש, כפי שנתבקש.

המסקנה המתוקנת – מאושרת מהקוד

התיקון שלך נכון, וחמור יותר ממה שהוא נשמע: המימוש הנכון כבר כתוב בקוד, ואף אחד לא קורא לו.

authUpgrade.acquire() שורות 161-165 בונה EmailAuthProvider.credential(email, password), ו-upgradeAnonymous כבר מבצע linkWithCredential על ה-anonymous user תוך שמירת ה-UID. הפער אינו חוסר במימוש – הוא בכך שמסלול המייל עוקף אותו לגמרי.

A. Call graph של EmailAuth היום

```
ContextualAuthSheet.attempt('email') ContextualAuthSheet.tsx:126
|- onCancel() <- הגיליון נסגר, pendingKind = null
\-- nav.navigate('EmailAuth')
    (!) return – upgradeAnonymous (שורה 135 לא מגיעה)
```

SignInScreen.tsx:239 -----> navigate('EmailAuth') [מסלול ישיר]

```
EmailAuthScreen.submit() EmailAuthScreen.tsx:101
|- mode === 'signIn' -> userStore.signInWithEmail
|   -> userService.signInWithEmail:351
|   -> fbSignInWithEmail (תקין – session מחליף)
|   -> set({ currentUser })
|   -> finishParkedAction() <- 085aa23 נוסף ב-
|
\-- mode === 'signUp' -> userStore.signUpWithEmail
    -> userService.signUpWithEmail:394
    -> fbSignUpWithEmail:224
    -> createUserWithEmailAndPassword (!) UID חדש
```

שני callers, הבדל מהותי ביניהם – זה בדיוק הפיצול C / B / A שביקשת להפריד.

B. איפה ה-UID anonymous הולך לאיבוד

נקודה אחת בדיוק: src/firebase/auth.ts:232

```
const cred = await fbCreateUserWithEmailAndPassword(auth, trimmed, password);
```

`createUserWithEmailAndPassword` יוצר חשבון חדש ומחליף את הסשן. ה-`anonymous user` לא מקושר ולא נמחק - ננטש. הוא נשאר ב-Firebase Auth כחשבון יתום, ומצטרף ל-416 החשבונות האנונימיים שכבר קיימים בפרודקשן.

מה נאבד יחד איתו

דבר	למה
attribution	applyInviteAttributionIfFresh / applyAcquisitionIfFresh (userService:427-428) רצים על ה-UID החדש, אז ייחוס שנקשר לסשן האנונימי לא עובר
bindUpgradedIdentity	לא נקרא כלל - כלומר אין <code>joryio.alias()</code> . לפי התיעוד שלנו <code>alias</code> הוא המנגנון היחיד שתופר את שתי הזהויות בשני ה-platforms; בלעדיו הריצה האנונימית מתנתקת מהמשתמש
acquired.mirror()	למייל זה קס-סו - אין הפסד כאן

C. איפה נכון לבצע linkWithCredential

לא צריך לכתוב את זה מחדש. `upgradeAnonymous('email', { email, password })` כבר מבצע את הרצף המלא:

שלב	שורה	מה קורה
acquire	161-165	EmailAuthProvider.credential(email, password)
Case A	197	linkWithCredential(anonUser, cred) - אותו UID
mirror + identity	198-199	acquired.mirror() + bindUpgradedIdentity(uid, 'email', true)
החזרה	200	{ status:'linked', uid, isNewAccount:true }
Case B (collision)	218-221	signInWithCredential → { status:'switched', isNewAccount:false }

הפונקציה גם לעולם לא זורקת - כל תוצאה היא ערך, בדיוק כדי שפעולה ממתינה לא תאבד. המקום הנכון לקרוא לה הוא `EmailAuthScreen.submit()`, בענף ה-`signUp`, כאשר ההקשר הוא `contextual upgrade`.

D. איך EmailAuthScreen יידע שזה contextual upgrade

AuthStack.tsx:10 מגדיר EmailAuth: undefined - אין params היום.

גישה	יתרון	סיכון
<code>{ context: route param מפורש 'contextual_upgrade' }</code>	מפורש, ניתן לבדיקה, מפריד A מ-B בלי ניחוש	דורש עדכון ה-param list בארבעה stacks
זיהוי <code>auth.currentUser.isAnonymous</code> ב-submit	אפס שינויי ניווט	heuristic שביר

ההמלצה: route param. הזיהוי האוטומטי שביר בדיוק כפי שחששת - מאז ה-silent guest כל משתמש לא-מחובר הוא אנונימי, כולל מי שנכנס דרך `SignInScreen`. ה-param הוא הדבר היחיד שמבדיל בין A ל-B.

פער רישום שצריך להיסגר יחד עם זה

`EmailAuth` רשום ב-AuthStack, ProfileStack, CommunitiesStack, GameStack - **לא ב-ChatStack**. אם גיליון האימות ייפתח מטאב הצ'אטים, `navigate('EmailAuth')` הוא `on-screen` שקט. זו אותה מחלקת באג שתוקנה ב-`06d1cae`.

E. success / collision / cancel / failure

מצב	outcome	UID	PendingAction	draft
link מצליח	<code>linked, isNewAccount:true</code>	נשמר	נשאר עד תוצאה סופית	נשאר
email כבר קיים	<code>failed, code email-already-in-use</code>	אנונימי נשאר	נשאר	נשאר
ביטול	<code>cancelled</code>	אנונימי נשאר	נשאר	נשאר
סיסמה שגויה ב-signIn	<code>throw → handleAuthError</code>	אנונימי נשאר	נשאר	נשאר

על `isAlreadyInUse` collision: מכניס ל-Case B, שמנסה `signInWithCredential` עם הסיסמה שהוזנה. אם היא אינה הסיסמה של החשבון הקיים, זה ייכשל. לכן הכוונה למסלול **Sign In** היא הנכונה - לא merge, לא ניחוש.

ממצא נוסף - `fetchSignInMethodsForEmail`

`src/firebase/auth.ts:246` משתמש ב-`fetchSignInMethodsForEmail` כדי להחליט אם להציע Google או Apple. אימתתי מול Firebase שה-project מריץ `enableImprovedEmailPrivacy: true` - כלומר הפונקציה מחזירה **תמיד**. `EmailRegisteredWithProviderError` לא נזרק בפועל, וההנחיה לספק הנכון מתה. זה נופל בדיוק תחת ההוראה שלך לא להסתמך על provider discovery.

F. exactly-once אחרי השינוי

המנגנון אינו תלוי ב-provider ולכן שורד כמו שהוא. שני מנעולים:

1. `inFlight: Set (actionCoordinator.ts:87)`, מופתח ב-`lockKey(kind, targetId)` - קריאה שנייה מחזירה `{ status: 'busy' }`

2. ניקוי רק בתוצאה עסקית סופית - `draftStore.consume()` + `clearPendingAction()` בשורות 284/290, מאחורי `terminal`

הסיכון היחיד: אם גם המסך וגם הגיליון יקראו ל- `resumePendingAction()`. המנעול מטפל בזה (השנייה מקבלת `busy`), אבל התכנון צריך שהמייל יקרא ל-`resume` בדיוק פעם אחת ולא ייצור מסלול מקביל - כפי שהורית.

G. Dead code

פריט	מצב	המלצה
<code>email branch (161-165)</code> <code>acquire()</code>	קיים, נכון, אפס <code>callers</code>	לחבר - זה המימוש הקנוני
<code>upgradeAnonymous(method, opts)</code> - הפרמטר <code>opts</code>	אף קורא לא מעביר אותו	יתחיל לשמש
<code>fetchSignInMethodsForEmail (auth.ts:246)</code>	מחזיר <code>[]</code> תמיד	להסיר את ההסתמכות
<code>EmailRegisteredWithProviderError</code>	נזרק רק מהענף המת שמעל	לבחון אחרי ההסרה

לא יישארו שני מימושים. `upgradeAnonymous` הופך ליחיד עבור המסלול ההקשרי, ו- `userService.signUpWithEmail` נשאר רק למסלול הישיר (A).

H. קבצים צפויים להשתנות

קובץ	שינוי	היקף
src/navigation/AuthStack.tsx	EmailAuth: { context?: ... }	שורה
src/navigation/ProfileStack.tsx	אותו param	שורה
src/navigation/CommunitiesStack.tsx	אותו param	שורה
src/navigation/GameStack.tsx	אותו param	שורה
src/navigation/ChatStack.tsx	רישום EmailAuth החסר	4~ שורות
src/components/auth/ContextualAuthSheet.tsx	להעביר 'contextual_upgrade' { context: 'contextual_upgrade' }	שורה
src/screens/auth/EmailAuthScreen.tsx	ענף upgradeAnonymous → signUp: contextual	30~ שורות
src/firebase/auth.ts	הסרת ההסתמכות על fetchSignInMethodsForEmail	12~ שורות
src/services/analyticsService.ts	אולי שדה upgrade_path ב-AuthCompleted	שורה
tests/...	קובץ חדש - 13 המקרים שביקשת	חדש

Google / Apple: אפס שינוי התנהגותי. הם כבר עוברים דרך upgradeAnonymous ואינם נוגעים באף קובץ מלבד רשימות ה-params.

שלוש נקודות שדורשות החלטה לפני מימוש

1. **profile completion אחרי linkWithCredential**. link מחזיר isNewAccount: true, ולכן onAuthenticated עושה return מוקדם וה-resume נדחה ל-boot pass שאחרי מסך הפרופיל. זו הזרימה הקיימת של Google/Apple ואשמר אותה - אבל המשמעות היא שהמשתמש יראה מסך פרופיל לפני שהמועדון נוצר. התנהגות מכוונת היום, ורציתי לוודא שהיא רצויה גם למייל.

2. getIdTokenResult(true) - לא אוסיף בלי ראיה. לפי §6 ביקשת להוכיח ולא לבטח. ההצעה: אחרי link לוודא ש-auth.currentUser.isAnonymous === false ושכתובה ל-/users/{uid} - ש-ה-rules מגדרים ב-isFullAccount() - מצליחה. אם היא מצליחה, ה-token כבר תקין ואין צורך ברענון. זה דורש הרצה מול Firebase אמיתי, לא mock.

3. **attribution**. היום היא רצה בתוך userService.signUpWithEmail על UID חדש. אחרי link ה-UID אינו משתנה, ולכן צריך להחליט היכן היא רצה - סביר שבנקודה שבה מסמך /users/{uid} נוצר לראשונה לאחר ה-link. זה החלק שאני הכי פחות בטוח לגביו.

- לא שונה קוד
- לא בוצע deploy
- לא שנו Firestore rules
- לא שונה `enableImprovedEmailPrivacy`
- לא בוצעה mutation ל-Auth בפרודקשן
- עצירה לפני מימוש, בהמתנה לאישור